ADVANCED COMPILER ENGINEERING

LLVM Kaleidoscope Compiler Architecture & Implementation

Comprehensive Deep-Dive into Token Rules, Quadruple/Triple IR, AST Design, Error Handling, Memory Segments, and Target Architecture

Repository: llvm-kaleidoscope

Backend: LLVM IRBuilder & Target Machine

Language: C++14 / Modern OOP

Focus: Frontend, IR Rules & Output Segments

CORE GOALS

Project Objectives

3 Core Pillars

01

1. Hand-Written Lexing & Operator-Precedence Parsing

02

2. Polymorphic AST Construction & Scoped Semantic Analysis

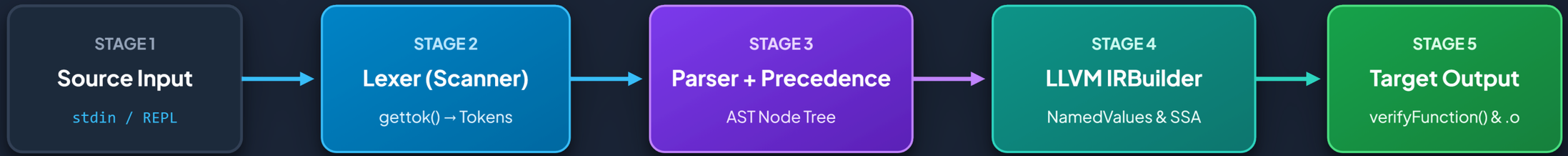
03

3. SSA LLVM IR Generation & Target Architecture Emission

SYSTEM ARCHITECTURE

High-Level Compiler Pipeline

End-to-End Pipeline



Interactive REPL Loop

Reads standard input continuously, classifies statements into definitions, externs, or immediate top-level expressions.

Verification & Segment Lowering

Emits validated LLVM byte-code into Module sections and dumps diagnostic text to standard output/error channels.

Complete Token Types & Lexer Rules

lexer/token.h & lexer.cpp

Token Enumeration & Value Mapping

Token Category	Enum Identifier	Value	Lexer Match Pattern
EOF Token	`tok_eof`	`-1`	End-of-File stream (`EOF`)
Command: Def	`tok_def`	`-2`	Keyword `"def"`
Command: Extern	`tok_extern`	`-3`	Keyword `"extern"`
Identifier	`tok_identifier`	`-4`	`[a-zA-Z][a-zA-Z0-9]*` → `IdentifierStr`
Numeric Literal	`tok_number`	`-5`	`[0-9.]+` → `NumVal (double)`
Binary Operators	ASCII direct	`[0-255]`	+ (43), - (45), * (42), < (60)
Delimiters	ASCII direct	`[0-255]`	((40),) (41), , (44), ; (59)
Comments	Ignored	N/A	`#.*[\r\n]` (Skipped to next token)

```
// Complete Token Definitions in lexer/token.h
enum Token {
  tok_eof = -1,
  // Commands
  tok_def = -2,
  tok_extern = -3,
  // Primary Entities
  tok_identifier = -4,
  tok_number = -5
};

// Scanning Rule in lexer/lexer.cpp
int gettok() {
  while (isspace(LastChar)) LastChar = getchar();
  if (isalpha(LastChar)) { /* def, extern, identifier */ }
  if (isdigit(LastChar) || LastChar == '.') { /* number */ }
  if (LastChar == '#') { /* skip comments */ return gettok(); }
  if (LastChar == EOF) return tok_eof;
  int ThisChar = LastChar; LastChar = getchar();
  return ThisChar; // return ASCII '+' / '(' / ';'
}
```

Quadruple & Triple Rules in Intermediate Code

4 Quadruple Rules: `(op, arg1, arg2, result)`

A 4-field tuple structure where the temporary result location is explicitly named:

Index	Operator (op)	Arg 1	Arg 2	Result (res)
(0)	FMUL	y	4.0	t1 (%multmp)
(1)	FADD	x	t1	t2 (%addtmp)
(2)	RET	t2	-	-

Advantage: Easy statement reordering and global optimization passes.

3 Triple Rules: `(op, arg1, arg2)`

A 3-field structure referencing intermediate values by their entry index:

Index	Operator (op)	Arg 1	Arg 2	Implied Value
(0)	FMUL	y	4.0	Points to (0)
(1)	FADD	x	(0)	Points to (1)
(2)	RET	(1)	-	Returns (1)

Advantage: Compact memory representation without declaring temporary names.

🎯 LLVM SSA IR as Generalized Quadruples

LLVM IR directly implements the Quadruple model in Static Single Assignment (SSA) form: `%addtmp = fadd double %x, %multmp` corresponds precisely to the Quadruple `(fadd, %x, %multmp, %addtmp)`.

Parser Architecture & Operator Precedence

parser/parser.cpp

Operator Precedence Climbing

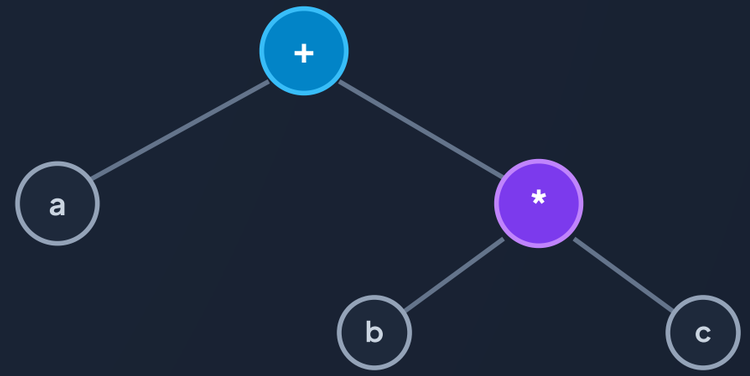
Uses recursive descent combined with operator precedence climbing for mathematical expressions.

Precedence Table

`<` : 10 | `+`, `-` : 20 | `*` : 40

- `ParsePrimary()` resolves numbers, variables, function calls, and parenthesized expressions.
- `ParseBinOpRHS()` dynamically groups subtrees based on comparative token precedence.

Expression Tree: `a + b * c`



Operator precedence ensures * binds tighter, forming the RHS subtree first.

INTERNAL REPRESENTATION

Abstract Syntax Tree (AST) Class Hierarchy

ast/ Directory

1 NumberExprAST

Numeric constants (e.g. `42.0`).

```
double Val;
codegen();
```

2 VariableExprAST

Variable reference (e.g. `x`).

```
string Name;
codegen();
```

3 BinaryExprAST

Binary operations (`LHS op RHS`).

```
char Op;
unique_ptr LHS, RHS;
```

4 CallExprAST

Function calls: `sin(x, y)`.

```
string Callee;
vector Args;
```

5 PrototypeAST

Captures function metadata: signature name and parameter identifiers list. Emits `llvm::Function*` declarations.

6 FunctionAST

Combines a `PrototypeAST` with a body `ExprAST`. Manages the function entry basic block and populates argument symbol tables.

Error Handling Based on All Token Rules

Token Type / Rule	Trigger Condition	Error Diagnostic Emitted	Compiler Recovery Action
`tok_identifier` (Prototype)	Missing function name after `def`/`extern`	`"Expected function name in prototype"`	Returns `nullptr` via `LogErrorP`
`'('` (Prototype Open)	Missing opening parenthesis in parameter list	`"Expected '(' in prototype"`	Returns `nullptr` via `LogErrorP`
`')'` (Prototype Close)	Missing closing parenthesis after parameters	`"Expected ')' in prototype"`	Returns `nullptr` via `LogErrorP`
`')'` (Paren Expression)	Unmatched closing parenthesis in `(expr)`	`"Expected )"`	Returns `nullptr` via `LogError`
`','` / `')'` (Call Args)	Missing comma or closing paren in argument list	`"Expected ')' or ',' in argument list"`	Aborts call node construction
`tok_identifier` (Scope)	Identifier not found in `NamedValues` symbol table	`"Unknown variable name"`	Returns `nullptr` via `LogErrorV`
Function Arity Rule	Argument count $\neq$ prototype parameter count	`"Incorrect # arguments passed"`	Aborts function call codegen
Codegen Failure Rule	Function body codegen returns `nullptr`	Transactional Module Rollback	`TheFunction->eraseFromParent()`

Output Channels & Target Memory Segments

1. Standard Output Streams

- **Standard Error (`stderr`):**
  - REPL interactive prompt: `ready>`
  - Diagnostics & Error Traps: `LogError: ...`
  - IR Emission feedback: `Read function definition:`
  - Final Module Dump: `TheModule->print(errs(), nullptr)`
- **Standard Input (`stdin`):**
  - Interactive command stream & file pipe execution (`./main < test.kld`)

2. Target Memory Segments

- **Text Segment (`.text` / Code):** Generated machine instructions for function basic blocks (e.g. `@foo`, `@__anon_expr`).
- **Read-Only Data (`.rodata` / Constant Pool):** Floating-point constants emitted via `ConstantFP::get(TheContext, APFloat(Val))`.
- **Symbol Table (`.symtab` / Relocations):** External declarations (`declare double @sin(double)`) and local symbol definitions.
- **Heap & Memory Arena:** AST ownership nodes managed via `std::unique_ptr` and LLVM IRContext allocation pools.

HARDWARE TARGET

Target Architecture & Native Binary Emission

LLVM TargetMachine & Object Files

🎯 Target Triples

Defines cross-platform target hardware configurations:

```
"x86_64-pc-linux-gnu"
"x86_64-w64-windows-msvc"
"aarch64-apple-darwin"
```

- `InitializeAllTargetInfos()`
- `InitializeAllTargets()`
- `InitializeAllAsmPrinters()`

⚙ TargetMachine & ABI

Configures CPU capabilities, ABI conventions, and memory alignments:

- **DataLayout:** Dictates 64-bit pointer size and memory alignments.
- **CPU Architecture:** Targets specific CPUs (e.g. `generic`, `haswell`).
- **Relocation Model:** Configured for Position Independent Code (`Reloc::PIC_`).

📦 Object File Emission

Lowers LLVM IR directly into binary object files (`.o` / `.obj`):

```
legacy::PassManager pass;
auto FileType = CGFT_ObjectFile;
TheTargetMachine→
  addPassesToEmitFile(pass, dest, nullptr, FileType);
pass.run(*TheModule);
```

Linkable via `clang/gcc/lld` into native executables.

CONCRETE TRACE & OUTPUT

Execution Trace & Output Segment Breakdown

`def foo(x y) x + y * 4.0;`

1. Tokens & Grammar

```
[tok_def] "def"
[tok_identifier] "foo"
['(']
[tok_identifier] "x"
[tok_identifier] "y"
[')']
[tok_identifier] "x"
['+']
[tok_identifier] "y"
['*']
[tok_number] 4.0
[';']
```

2. Quadruple / AST Lowering

```
// Quadruple Form
(0) (FMUL, y, 4.0, %multmp)
(1) (FADD, x, %multmp, %addtmp)
(2) (RET, %addtmp, -, -)

// Scoping Check
NamedValues["x"] = %x
NamedValues["y"] = %y
```

3. Emitted Output & Segment

```
// Channel: stderr
Read function definition:
// Segment: .text (Code)
define double @foo(double %x, double %y) {
entry:
  %multmp = fmul double %y, 4.000000e+00
  %addtmp = fadd double %x, %multmp
  ret double %addtmp
}
```

ENGINEERING ROADMAP

Future Work Plan & Implementation Milestones

Work Plan Diagram

P1 Phase 1: JIT & Optimization

Milestone 1 • Chapter 4

- Integrate **LLVM OrcJIT / LLJIT** to evaluate top-level expressions immediately.
- Attach `FunctionPassManager` with GVN and Instruction Combining passes.

P2 Phase 2: Control Flow & State

Milestone 2 • Chapters 5–7

- Implement `IfExprAST` with conditional phi-nodes.
- Add `ForExprAST` loop construct.
- Add mutable local variables with stack `alloca` and LLVM `mem2reg`.

P3 Phase 3: Target Object Emission

Milestone 3 • Chapters 8–9

- Configure target machine triple and data layouts.
- Emit native object code (`.o` / `.obj`) and link to standalone executables.
- Add DWARF / CodeView debug symbols.

🎉 SUMMARY & WRAP-UP

Thank You! Questions & Discussion

Key Takeaways:

- Clear separation of concerns between Tokenizer, Grammar Parser, AST, and LLVM Backend.
- Comprehensive token-rule-based error handling ensuring graceful REPL recovery and transactional IR rollbacks.
- Production-grade lowering to verified SSA Quadruple IR and clear target architecture mapping for native binary compilation.

📖 **Reference:** llvm.org/docs/tutorial

📁 **Repository:** llvm-kaleidoscope